Lab 5 — React Polymorphism & Niche Product CRUD

1. The Niche Product

My niche product is a board game. `BoardGameEntity` extends `PublicationEntity` because a board game is sold the same way a publication is (it has a title, a price and a few copies) so it inherits those three fields instead of declaring them again.

Everything is still stored in the one `product` table the project already used. Each row is told apart by a discriminator column, and mine is `BOARDGAME`.

Fields

Field	Java type	JSON type	Comes from	Form input
`designer`	String	string	BoardGameEntity	text
`minPlayers`	Integer	number	BoardGameEntity	number
`maxPlayers`	Integer	number	BoardGameEntity	number
`playTimeMinutes`	Integer	number	BoardGameEntity	number
`cooperative`	Boolean	boolean	BoardGameEntity	checkbox
`releaseDate`	LocalDate	string, yyyy-MM-dd	BoardGameEntity	date picker

Every field a user can fill in uses a wrapper type (`Double`, `Integer`, `Boolean`). If I had written `boolean cooperative` and the browser sent `"cooperative": null`, the whole request would come back as a 400. The same constructor also needs `@JsonProperty` names on its parameters, exactly like the magazine one, because Jackson matches the JSON to the constructor by parameter name.

API

Verb	Path	Sends	Returns	Errors
GET	`/api/rest/boardgames`	—	200, a list of games	—
GET	`/api/rest/boardgames/{id}`	—	200, one game	404
POST	`/api/rest/boardgames`	a game	200, the saved game with its new id	—
PUT	`/api/rest/boardgames/{id}`	a game	200, the updated game	404
DELETE	`/api/rest/boardgames/{id}`	—	200, empty body	404, 409

The identifier is `id`. When an id does not exist the API answers 404 with a plain-text message instead of an error page. I did not write a new handler for that — I made `BoardGameNotFoundException` extend `ResourceNotFoundException`, so it reuses the one I already wrote in Lab 4.

Deleting goes through `ProductDeletionService`, which takes the game out of any shopping cart first. Without that step, deleting a product that is in a cart fails. If the game has already been bought, the service refuses and the API answers 409 instead.

What the response looks like

```
GET /api/rest/boardgames/8


{
  "title": "Mysterium",
  "price": 49.99,
  "copies": 4,
  "designer": "Oleg Sidorenko, Oleksandr Nevskiy",
  "minPlayers": 2,
  "maxPlayers": 7,
  "playTimeMinutes": 42,
  "cooperative": true,
  "releaseDate": "2015-10-01",
  "id": 8,
  "productId": "SKU-1007",
  "productType": "BoardGameEntity"
}
```

2. React Implementation

Files added or changed

File	What it does
`src/BoardGame.jsx`	One row in the list. Clicking Edit turns the same row into a small form.
`src/BoardGameList.jsx`	Shows the rows. It has no data of its own — it just displays what it is given.
`src/AddBoardGameForm.jsx`	The form for adding a new game. It sends the POST itself.
`src/App.jsx`	Changed. Holds the list of games, loads them, and handles update and delete. Also declares my two routes.
`src/NavBar.jsx`	Changed. Two new links, next to the Book and Magazine ones.
`src/App.css`	Changed. Styles for the new rows and the checkbox label.

Route	What it shows
`/boardgames`	The list of games, each with Edit and Delete
`/boardgames/add`	The add form

How the pieces talk to each other

The rule is that data flows down and events flow back up.

- **Data down.** `App` gives `BoardGameList` the games plus a loading flag and an error message. That component gives each `BoardGame` one game and the two callbacks. Nothing ever changes a prop it was handed.
- **Events up.** A row calls `onDelete(id)` or `onUpdate(id, data)`; the form calls `onBoardGameAdded(saved)`. The child asks the parent to change the list — it never edits the list itself.
- **Loading the data.** A `useEffect` with an empty dependency array runs the first GET when the app mounts. Its cleanup sets a flag so that if the component disappears before the response arrives, the old response is ignored instead of updating something that is no longer on screen.
- **Updating the list.** Adding uses `[...previous, saved]`, editing uses `map` to replace just the one matching game, and deleting uses `filter`. If I used `push` the array would be the same object as before, React would not notice anything changed, and the screen would not update.

3. Testing

Build and tests

```
$ npm run lint
  (no output, exit 0)

$ npm run build
  vite v8.3.6 building client environment for production...
  ✓ 105 modules transformed.
  dist/index.html                   0.45 kB | gzip:  0.29 kB
  dist/assets/index-CSgv7TMj.css    1.07 kB | gzip:  0.54 kB
  dist/assets/index-CnGGKvIF.js   272.13 kB | gzip: 84.12 kB
  ✓ built in 825ms

$ ./mvnw test -Dspring.profiles.active=h2
  9 tests, 0 failures, 0 errors
```

Create, read, update, delete

After every change I asked the server for the record again. That is the only way to know the data really saved. This is a page reload and a fresh request, I did not restart the database.

```
POST   /api/rest/boardgames      -> 200   created id=13
GET    /api/rest/boardgames/13   -> 200   cooperative:false  playTimeMinutes:75

PUT    /api/rest/boardgames/13   -> 200   (edited in the browser)
GET    /api/rest/boardgames/13   -> 200   title "Lab 5 Verification Game (updated)"
                                          playTimeMinutes 95   (number)
                                          cooperative     true (boolean)
                                          price           64.5 (number)
                                          releaseDate     "2026-03-14"

DELETE /api/rest/boardgames/13   -> 200   (empty body, so I do not parse JSON)
GET    /api/rest/boardgames/13   -> 404   Could not find board game with ID: 13

PUT    /api/rest/boardgames/9999 -> 404   Could not find board game with ID: 9999
DELETE /api/rest/boardgames/9999 -> 404   Could not find board game with ID: 9999
GET    /api/rest/boardgames/1    -> 404   (id 1 is a Book, so it is not found here)
```

When a request fails

To check the error handling I made every PUT fail with a 503, then tried to rename Mysterium to “Mysterium SHOULD NOT SAVE”. Three things happened:

- the edit form stayed open with my typed text still in it;
- `Update failed (503)` appeared under the form;
- the saved game did not change — pressing Cancel brought back the real Mysterium, and asking the server again still returned the original title.

A failed add behaves the same way (the form keeps what was typed), and a failed delete leaves the row where it is.

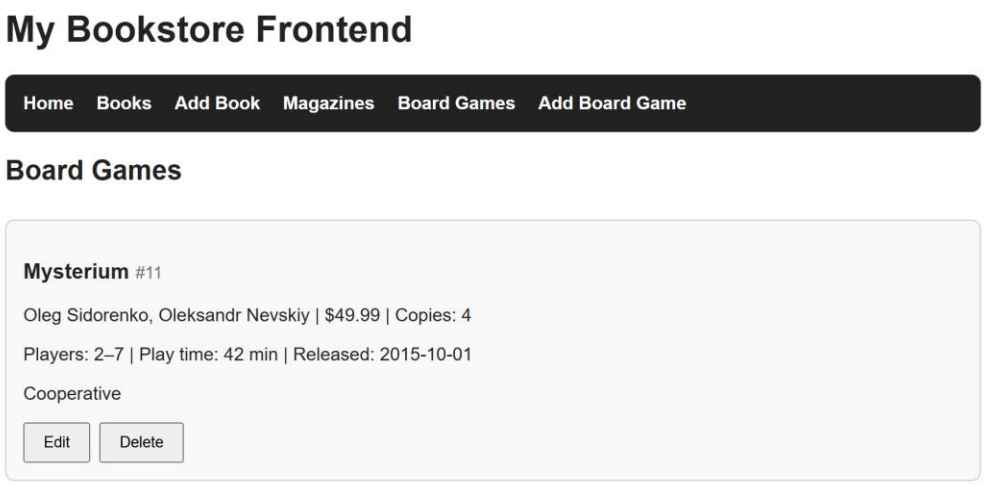
The rest of the project still works

Adding my niche product did not break anything that was already there:

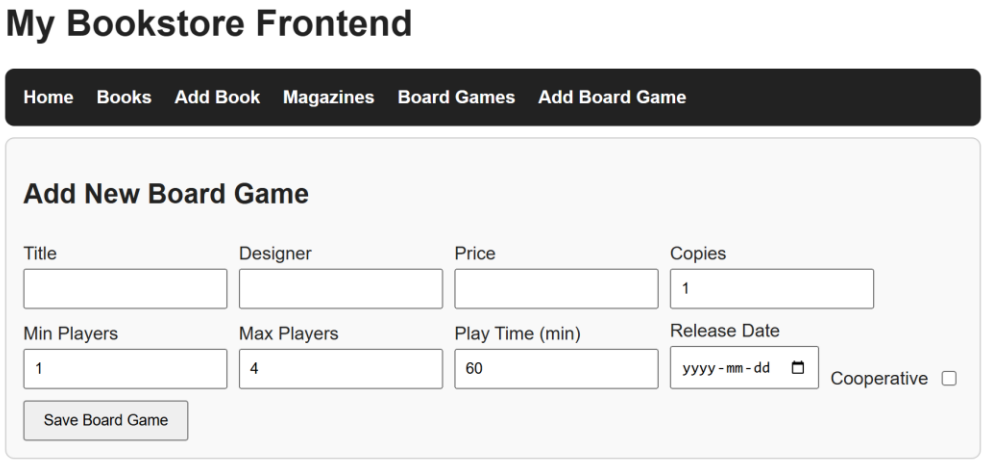
Check	Result
Opening /books without logging in	302, redirected to /login
The login page	200
Logging in through the JWT endpoint	200, returns a token
Swagger UI	200, now lists 6 REST groups
Java tests	9 tests, 0 failures

4. Screenshots

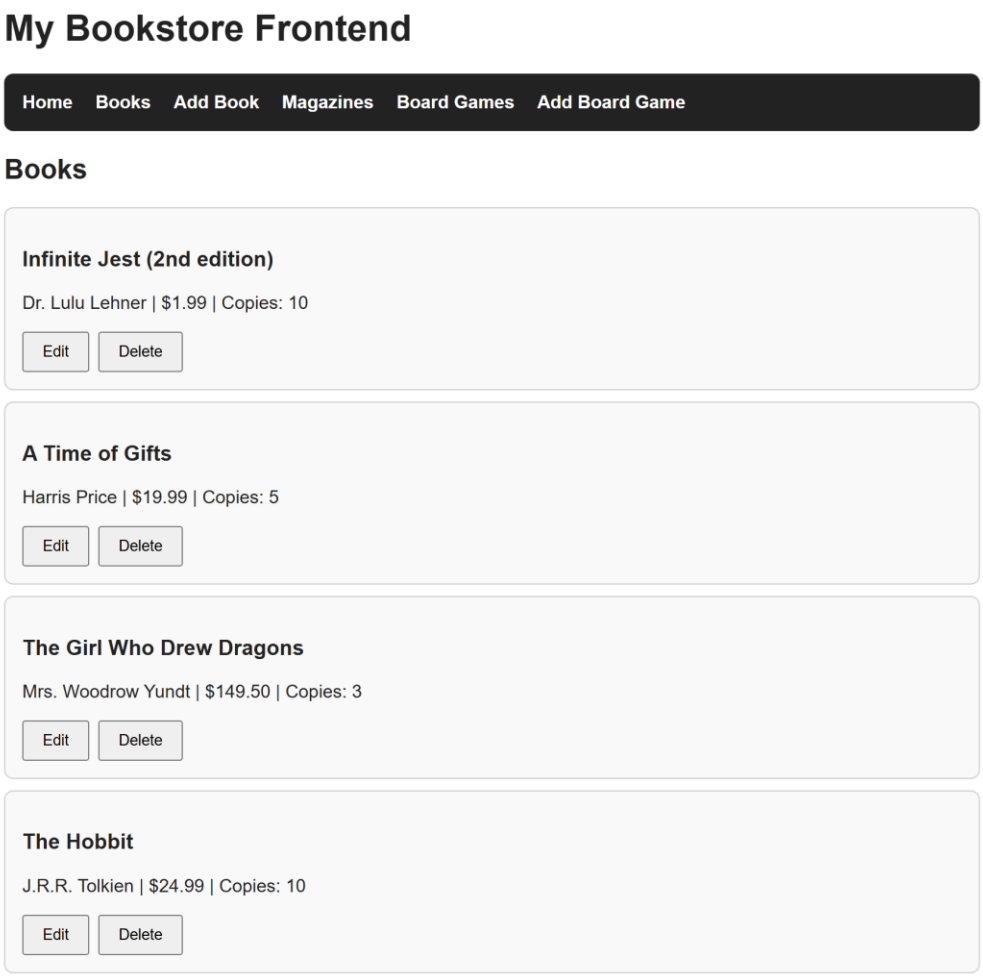
Board game list — http://localhost:5173/boardgames



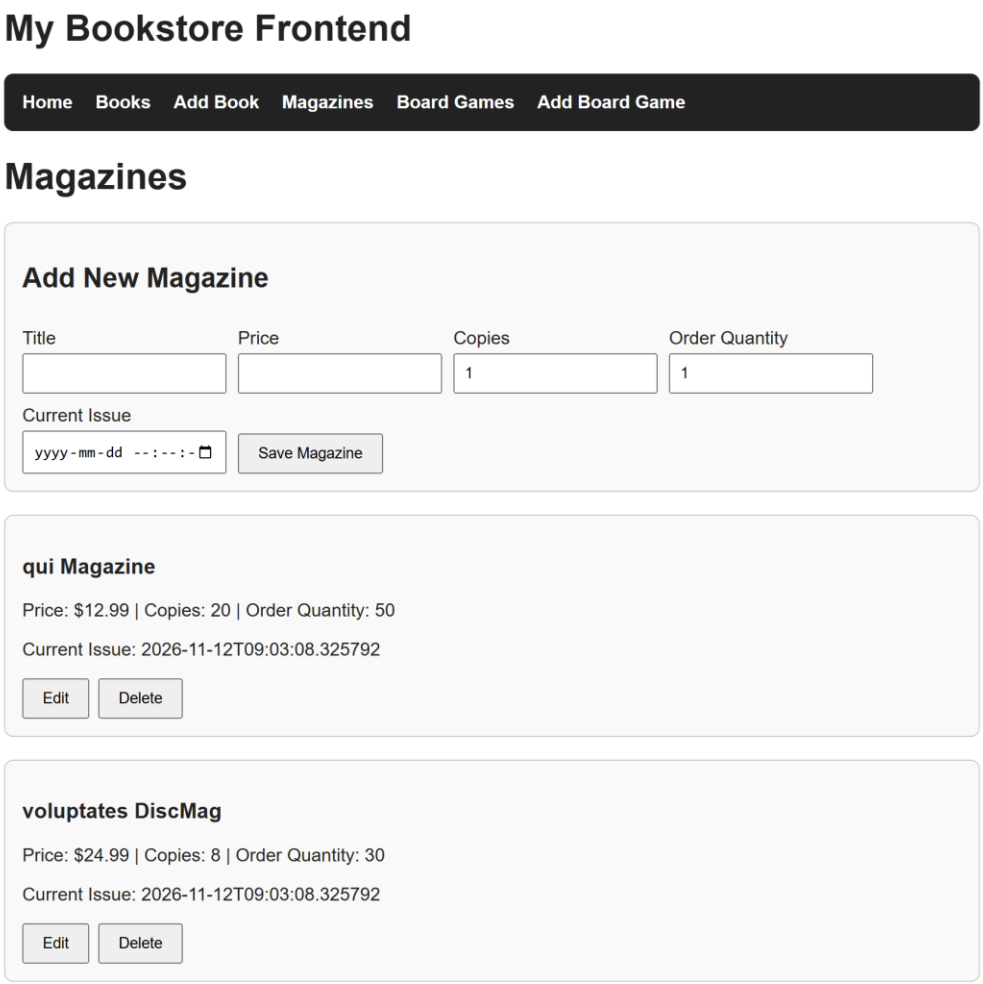
Add form — http://localhost:5173/boardgames/add



Book routes still work — /inventory



Magazine route still works — /magazines



My answers to the 50 multiple-choice questions are in the separate file *Multiple Choice Questions.pdf*.